Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Кафедра информатики и вычислительной техники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
дисциплины «Объектно-ориентированное программирование»
Вариант 12

Выполнила:
Коробка В.А.,
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент кафедры информатики и вычислительной техники института информационных технологий и кибербезопасности

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Элементы объектно-ориентированного программирования в языке Python

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Порядок выполнения работы

В ходе лабораторной работы был изучен теоретический материал, создан и клонирован публичный репозиторий на GitHub с лицензией MIT. В созданном проекте были проработаны все примеры лабораторной работы.

Условие индивидуального задания по варианту 12:

Парой называется класс с двумя полями, которые обычно имеют имена first и second. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать: метод инициализации __init__ ; метод должен контролировать значения аргументов на корректность; ввод с клавиатуры read ; вывод на экран display .

Реализовать внешнюю функцию с именем make_тип() , где тип — тип реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

В раздел программы, начинающийся после инструкции if __name__ = '__main__': добавить код, демонстрирующий возможности разработанного класса.

Для выполнения первого индивидуального задания был реализован класс Pair, содержащий два поля: first и second. Метод __init__ выполняет инициализацию объекта и проверку корректности переданных значений. Метод read обеспечивает ввод данных с клавиатуры, а метод display – вывод на экран.

Метод distance вычисляет расстояние от начала координат до точки, заданной полями first и second. Для создания объекта также реализована внешняя функция make_pair, которая проверяет переданные аргументы.

В основной части программы продемонстрированы создание объектов, ввод и вывод данных, вычисление расстояния и обработка некорректных аргументов.

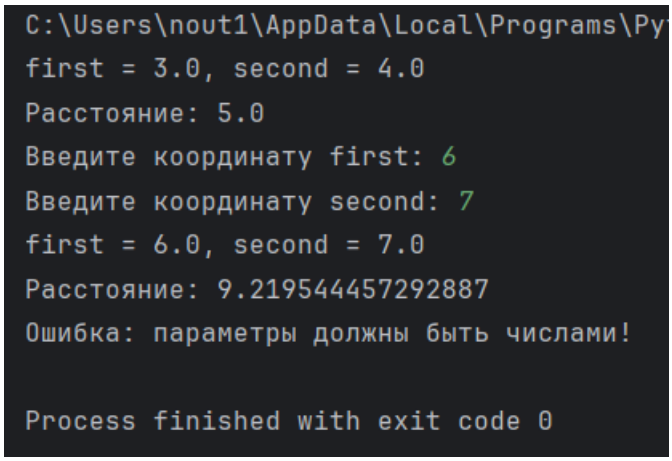


Рисунок 1. Результат выполнения индивидуального задания 1

Полный код задания:

```
import math
import sys



class Pair:
    def __init__(self, first, second):
        if isinstance(first, (int, float)) and isinstance(second, (int, float)):
            self.first = float(first)
            self.second = float(second)
        else:
            raise TypeError


    def read(self):
```

```
        self.first = float(input("Введите координату first: "))
        self.second = float(input("Введите координату second: "))

    def display(self):
        print(f"first = {self.first}, second = {self.second}")

    def distance(self):
        return math.sqrt(self.first ** 2 + self.second ** 2)


def make_pair(first, second):
    if isinstance(first, (int, float)) and isinstance(second, (int, float)):
        return Pair(first, second)
    else:
        print("Ошибка: параметры должны быть числами!")
        sys.exit()


if __name__ == '__main__':
    p1 = make_pair(3, 4)
    p1.display()
    print(f"Расстояние: {p1.distance()}")

    p2 = Pair(0, 0)
    p2.read()
    p2.display()
    print(f"Расстояние: {p2.distance()}")
```

p3 = make_pair("abc", 5)Условие индивидуального задания 2:

Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы: метод инициализации __init__ ; ввод с клавиатуры read ; вывод на экран display .

В раздел программы, начинающийся после инструкции if __name__ = '__main__': добавить код, демонстрирующий возможности разработанного класса.

12. Создать класс Fraction для работы с дробными числами. Число должно быть представлено двумя целочисленными полями: целая часть и дробная часть. Реализовать арифметические операции сложения, вычитания, умножения и операции сравнения.

Был реализован класс Fraction для работы с дробными числами. Число представлено двумя целочисленными полями: integer – целая часть и fractional – дробная часть. Дополнительно используется атрибут places, хранящий количество цифр после запятой.

Метод __init__ выполняет инициализацию объекта и проверку входных данных. Метод read обеспечивает ввод числа с клавиатуры, метод display – вывод числа на экран.

Арифметические операции сложения, вычитания и умножения реализованы с помощью методов __add__, __sub__ и __mul__. Для сравнения чисел используются методы __eq__, __ne__, __lt__, __gt__, __le__ и __ge__.

Для выполнения вычислений используются целочисленные операции, что позволяет избежать погрешностей, связанных с использованием типа float. В основной части программы демонстрируются арифметические операции, сравнение объектов, ввод и вывод числа.

```
PS C:\Users\nout1\Documents\university\sem5\oop\lab2> uv run python tasks/individual
2.py
a = 1,5
b = 2,25
a + b = 3,75
a - b = -0,75
a * b = 3,375
a == b: False
a != b: True
a < b: True
a > b: False
a <= b: True
a >= b: False
Целая часть: 2
Дробная часть: 03
Введено: 2,03
PS C:\Users\nout1\Documents\university\sem5\oop\lab2>
```

Рисунок 2. Результат выполнения задания 2

Полный код задания:

```
class Fraction:
    def __init__(self, integer=0, fractional="0"):
        text = str(fractional)
        digits = text.removeprefix("-")

        if type(integer) is not int or not digits.isdigit():
            raise ValueError("Введите целые числа")
        if integer > 0 and text.startswith("-"):
            raise ValueError("Знаки частей должны совпадать")

        self.integer = integer
        self.fractional = int(digits)
        if integer < 0 or text.startswith("-"):
            self.fractional = -self.fractional
        self.places = len(digits)
```

```
def read(self):
    integer = int(input("Целая часть: "))
    fractional = input("Дробная часть: ")
    self.__init__(integer, fractional)

def display(self):
    number = self._value(self.places)
    sign = "-" if number < 0 else ""
    digits = str(abs(self.fractional)).zfill(self.places).rstrip("0") or "0"
    print(f"{sign}{abs(self.integer)},{digits}")

def _value(self, places):
    number = self.integer * 10**self.places + self.fractional
    return number * 10 ** (places - self.places)

def _make(self, number, places):
    result = Fraction()
    sign = -1 if number < 0 else 1
    result.integer = sign * (abs(number) // 10**places)
    result.fractional = sign * (abs(number) % 10**places)
    result.places = places
    return result

def __add__(self, other):
    places = max(self.places, other.places)
    return self._make(self._value(places) + other._value(places), places)

def __sub__(self, other):
```

```
        places = max(self.places, other.places)
        return self._make(self._value(places) - other._value(places), places)

    def __mul__(self, other):
        places = self.places + other.places
        return self._make(self._value(self.places) * other._value(other.places),
places)

    def __eq__(self, other):
        places = max(self.places, other.places)
        return self._value(places) == other._value(places)

    def __ne__(self, other):
        return not self == other

    def __lt__(self, other):
        places = max(self.places, other.places)
        return self._value(places) < other._value(places)

    def __gt__(self, other):
        return other < self

    def __le__(self, other):
        return not self > other

    def __ge__(self, other):
        return not self < other
```

```
if __name__ == "__main__":
    a = Fraction(1, 50)
    b = Fraction(2, 25)

    print("a =", end=" ")
    a.display()
    print("b =", end=" ")
    b.display()
    print("a + b =", end=" ")
    (a + b).display()
    print("a - b =", end=" ")
    (a - b).display()
    print("a * b =", end=" ")
    (a * b).display()

    print("a == b:", a == b)
    print("a != b:", a != b)
    print("a < b:", a < b)
    print("a > b:", a > b)
    print("a <= b:", a <= b)
    print("a >= b:", a >= b)

    c = Fraction()
    c.read()
    print("Введено:", end=" ")
    c.display()
```

Для автоматического тестирования индивидуальных заданий использовалась библиотека pytest. В файле test_lab2.py реализовано восемь тестов для проверки работы классов Pair и Fraction.

Тесты проверяют вычисление расстояния, арифметические операции над дробными числами, операции сравнения, работу с отрицательными числами и вычисления с большим количеством знаков после запятой.

Тестирование выполнялось командой: uv run python -m pytest -v

В результате все восемь тестов были успешно пройдены. Для проверки качества и оформления кода использовался инструмент Ruff. Проверка индивидуальных заданий и тестов выполнялась командой: uv run ruff check tasks test_lab2.py

Дополнительно использовался инструмент pre-commit, позволяющий автоматически проверять код перед созданием коммитов. Примеры были исключены из проверки.

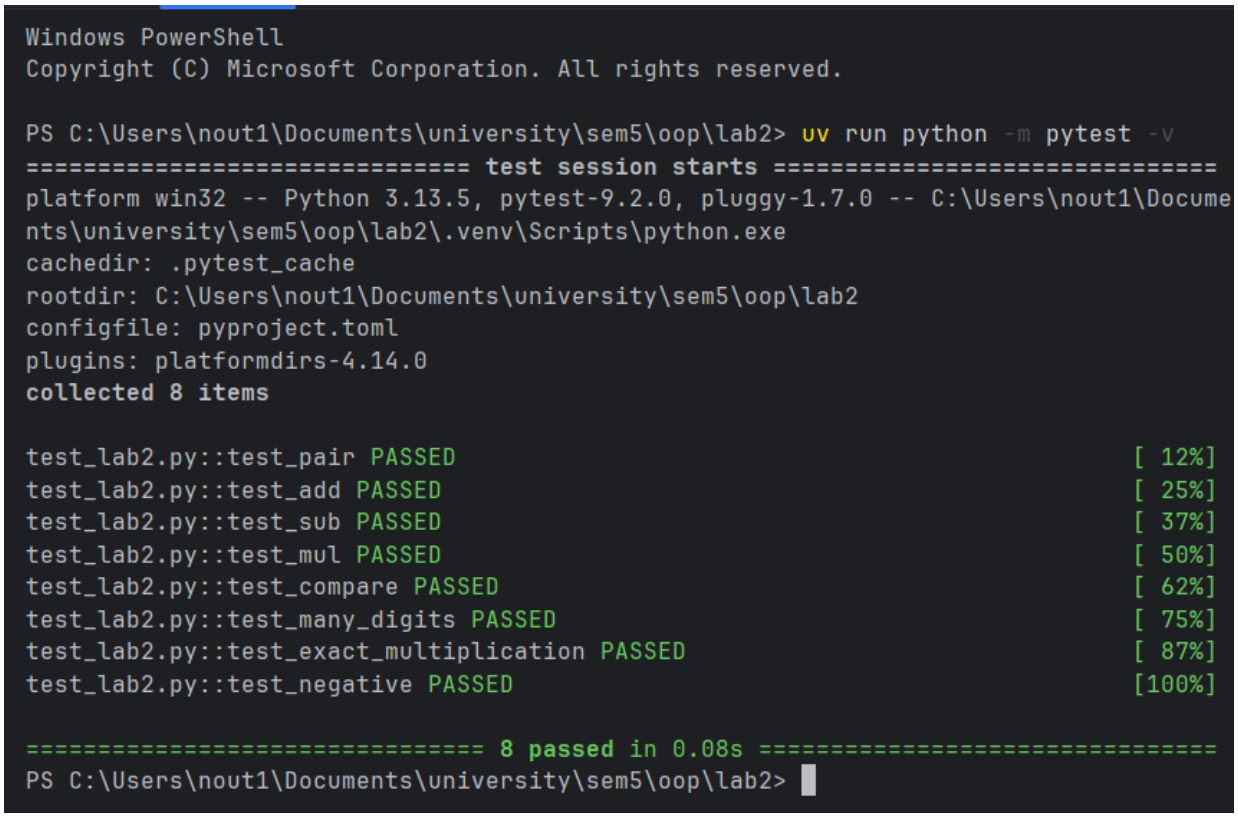


Рисунок 3. Результаты автоматического тестирования pytest

Ответы на контрольные вопросы

1. Как осуществляется объявление класса в языке Python?

Классы объявляются с помощью ключевого слова class и имени класса. Как правило, имя класса начинается с заглавной буквы и обычно является существительным или словосочетанием.

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

Атрибуты класса являются общими для всех объектов класса, а атрибуты экземпляра специфическими для каждого экземпляра. Атрибуты класса определяются внутри класса, но вне каких-либо методов, а атрибуты экземпляра обычно определяются в методах, чаще всего в __init__.

3. Каково назначение методов класса?

Методы определяют функциональность объектов, принадлежащих конкретному классу.

4. Для чего предназначен метод __init__() класса?

Метод __init__() является конструктором. Конструкторы – это концепция объектно-ориентированного программирования. Метод __init__() указывает, какие атрибуты будут у экземпляров нашего класса.

5. Каково предназначение self?

Аргумент self представляет конкретный экземпляр класса и позволяет нам получить доступ к его атрибутам и методам.

6. Как добавить атрибуты в класс?

Атрибуты класса добавляются путём присваивания значений внутри тела класса, но вне методов. Атрибуты экземпляра добавляются внутри методов, чаще всего в __init__, через обращение к self.

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

Атрибуты и методы, имя которых начинается с одного подчёркивания, считаются защищёнными и не предназначены для использования извне.

Атрибуты с двойным подчёркиванием являются приватными и подвергаются искажению имён. Для контролируемого доступа к приватным атрибутам используются декораторы @property и @имя_атрибута.setter.

8. Каково назначение функции isinstance?

Функция isinstance проверяет, принадлежит ли объект указанному типу или классу.

Вывод: В ходе выполнения лабораторной работы были изучены основные принципы объектно-ориентированного программирования в языке Python и приобретены практические навыки работы с классами и объектами. В процессе выполнения индивидуальных заданий были реализованы классы Pair и Fraction, в которых использовались методы инициализации с валидацией данных, ввод и вывод информации, а также перегрузка арифметических операторов и операторов сравнения.